TP1- Recorte consciente de audio

Fernandez Melia Bautista - Magni Manuel - Quiroga Gonzalo

Nota: Todas las observaciones hechas por los docentes fueron corregidas

1. Introducción

Contexto del problema En la producción musical y audiovisual, acortar una grabación para que entre en una duración determinada es una tarea cotidiana. Sin embargo, los enfoques más sencillos suelen tener limitaciones importantes: truncar la pista significa perder el cierre original de la pieza, mientras que acelerar la reproducción de forma uniforme altera el tempo y la altura de todo el material, lo que resulta inaceptable para el oído si se supera cierto factor.
Frente a estas limitaciones, surge la técnica del recorte consciente de contenido. Este enfoque propone no comprimir la pieza entera, sino aprovechar que la música tiene una estructura repetitiva (como las estrofas y estribillos) para descartar fragmentos completos y unir los extremos. La idea central es preservar la estructura de la pieza, sus estrofas, estribillos y cierres. Si se logra que un salto caiga en un punto donde el material que sigue es muy parecido al que debería haber sonado de forma natural, el corte resulta inaudible, una técnica que se utiliza desde hace tiempo en la producción de los *radio edits*.

Objetivo del trabajo El principal objetivo de este trabajo práctico es resolver el problema de optimización que tiene el recorte consciente, encontrando la selección de fragmentos que minimice el salto acústico percibido. Para lograrlo, se implementarán y analizarán tres algoritmos vistos en clase en el lenguaje C++: Fuerza Bruta, Backtracking y Programación Dinámica. Además, se comparará el rendimiento entre C++ y Python al implementar las soluciones de Backtracking y Programación Dinámica también en Python. Finalmente, se buscará establecer cuál es la recomendación definitiva sobre qué algoritmo y lenguaje utilizar en un escenario práctico en base a un análisis de rendimiento y experimentación.

Estructura del informe
El informe está organizado de la siguiente manera: en la sección de **Modelado**, se plantea matemáticamente el problema del recorte consciente y sus restricciones. Luego, en **Algoritmos implementados**, se muestra el diseño, el pseudocódigo, las podas (en el caso de Backtracking) y la complejidad de cada solución planteada. En la sección de **Experimentación**, se describe la metodología de testing y los datasets propios utilizados. Por otro lado, en **Resultados y discusión**, se encuentra el análisis del rendimiento de las distintas implementaciones. Por último, se cierra con una **Conclusión** que sintetiza los descubrimientos y hallazgos y presenta posibles mejoras.

2. Modelado

Para abordar el problema computacionalmente, se abstrae la grabación de audio y sus propiedades acústicas en un modelo matemático.
Se considera que una grabación completa se representa como una secuencia de n pulsos. Cada pulso i (donde $1 \leq i \leq n$) tiene asociado un vector de características $f_i \in R^d$. Este vector resume el contenido armónico y tímbrico del audio en ese instante específico.
El recorte del audio requiere que se conserven exactamente k pulsos. Se define una selección como una secuencia de índices que cumple con esta condición: $1 = i_1 < i_2 < \ldots < i_k = n$. Esta restricción formaliza tres reglas fundamentales para que la pista resultante tenga sentido:

1. El recorte siempre debe empezar en el primer pulso de la grabación original ($i_1 = 1$).
2. El recorte siempre debe terminar en el último pulso ($i_k = n$).
3. La pista siempre avanza hacia adelante temporalmente, sin repetir fragmentos, lo que se garantiza al exigir que los índices sean estrictamente crecientes.

La métrica para evaluar la calidad de un empalme es el costo de transición. Si se está reproduciendo el pulso i y se decide saltar a continuación al pulso j, se define el costo $c(i, j)$ como la distancia euclidiana entre el pulso que lógicamente debía seguir y el elegido a poner:
$c(i, j) = ||f_{i+1} - f_j||_2$

Esta función mide qué tan distinto es el pulso al que se salta respecto del que debería haber sonado a continuación. Si no se realiza ningún salto y se pasa al pulso siguiente de forma natural ($j = i + 1$), la distancia es contra sí mismo y el costo es 0.
Finalmente, el costo total (C) de la selección de audio se define como la sumatoria de todos los costos de transición individuales entre los k pulsos elegidos:

$$C = \sum_{t=1}^{k-1} c(i_t, i_{t+1})$$

El objetivo final del problema es encontrar la selección válida de k pulsos que devuelva el costo total C más bajo posible.

3. Algoritmos Implementados

3.1 Fuerza Bruta

El enfoque de fuerza bruta consiste en explorar el espacio de soluciones de manera exhaustiva, generando todas las combinaciones posibles de pulsos para encontrar la selección óptima. El algoritmo se estructura de forma recursiva, decidiendo en cada paso si un pulso i (desde 1 hasta n) se incluye o no en la secuencia actual.

Para garantizar que la solución construida sea válida respecto a las restricciones del problema, se fuerza siempre la inclusión del primer y el último pulso, es decir, i=1 e i=n. Una vez que el índice de exploración supera la cantidad total de pulsos n, la recursión alcanza su caso base y se verifica si la selección contiene exactamente k pulsos. En caso afirmativo, se calcula el costo total de la selección iterando sobre los pulsos elegidos y sumando los costos individuales c(i, j) de las transiciones entre pulsos consecutivos. Si este costo resulta ser estrictamente menor que el mejor costo registrado hasta el momento, se actualiza el mínimo global y se almacena la selección actual. Al finalizar la exploración completa del árbol de recursión, se garantiza la obtención de la solución exacta de mínimo costo.

Pseudocodigo FUERZA BRUTA

```
FB(S, i ∈ N)
if i > n then
      if |S| = k then
            costo_actual ← 0
            for t = 1, ..., |S| - 1 do
                  costo_actual ← costo_actual + costo(S[t], S[t+1])
            end for

            if costo_actual < mejorCosto then
                  mejorCosto ← costo_actual
                  B ← S
              end if
      end if
else
      obligatorio ← (i = 1 ∨ i = n)
      if no obligatorio then
            FB(S, i + 1);        ► Corresponde a no agregar el pulso i
      end if

      Agregar i al final de S;
      FB(S, i + 1);               ► Corresponde a agregar el pulso i
      Eliminar i de S;                 ► Deshace el cambio
end if
```

Orden de complejidad de la función de Fuerza Bruta:
En el caso base (cuando i > n), la evaluación de las condiciones lógicas y la inicialización de variables se ejecutan en tiempo constante O(1). Si la selección alcanza el tamaño requerido (|S| = k), el algoritmo ingresa a un ciclo que itera k-1 veces. Dentro de este ciclo, se calcula el costo evaluando la distancia euclidiana entre dos vectores de características de dimensión d, lo cual toma un tiempo O(d). Por consiguiente, aplicando la regla del producto de funciones, el costo total de este ciclo resulta en O(k * d). En seguida, la operación lógica que compara si el

costo actual mejora el óptimo global toma O(1). De cumplirse esta condición, se realiza una copia profunda del vector hacia la variable que almacena la mejor solución, añadiendo un costo de O(k). El tiempo de ejecución en el caso base queda dominado por la sumatoria de las distancias, consolidando un costo de O(k * d).
Para el caso general (cuando i <= n), determinar si el pulso actual es de inclusión obligatoria toma O(1). Posteriormente, el algoritmo despliega el árbol de búsqueda realizando hasta dos llamadas recursivas por cada pulso evaluado: una descartando el índice y otra incluyéndolo en la selección (las operaciones para insertar y retirar el pulso de la estructura se realizan en O(1)). Esta bifurcación constante genera un árbol de recursión de altura n con un crecimiento de O(2^n) ramas. Al multiplicar la cantidad de escenarios explorados por el trabajo realizado en los nodos hoja, la complejidad temporal del algoritmo queda O(2^n * k * d).

3.2 Backtracking

El algoritmo de Backtracking mejora la búsqueda exhaustiva construyendo la solución de manera incremental e interrumpiendo tempranamente las ramas de recursión que no pueden conducir a una solución óptima o válida. Para lograr esto, se lleva un registro del último pulso incluido y del costo parcial acumulado, implementando dos estrategias de poda fundamentales.

Por un lado, la poda por optimalidad evalúa en cada paso si el costo parcial actual supera o iguala al mejor costo registrado globalmente. Como los costos de transición son aditivos y no negativos, se descarta la rama inmediatamente ya que es imposible mejorar el óptimo encontrado.

Por otro lado, la poda por factibilidad verifica la cantidad de pulsos seleccionados respecto al tamaño k deseado. Si ya se seleccionaron más de k pulsos (faltan < 0), o si los pulsos que quedan por evaluar en la grabación no alcanzan para completar los k requeridos, la exploración se detiene por ser una solución lógicamente imposible.

Si el estado supera ambas podas, el algoritmo ramifica evaluando la inclusión o exclusión del pulso actual, respetando siempre la obligatoriedad del primer y último pulso. Al calcular y acumular el costo de transición paso a paso durante la inclusión, el caso base simplemente actualiza el mínimo global sin necesidad de recalcular toda la sumatoria de distancias.

Pseudocodigo BACKTRACKING

```
BT(S, i, costo_actual)

if costo_actual ≥ mejor_costo then                    ► Poda por optimalidad

    return

end if
```

```
if i > n then                                                    ▶ Caso base
    if |S| = k then
        mejor_solucion ← S
        mejor_costo ← costo_actual
    end if
    return
end if
faltan ← k - |S|
restantes ← n - i + 1
if faltan < 0 ∨ faltan > restantes then                ▶ Poda por factibilidad
    return
end if
if i ≠ 1 ∧ i ≠ n then
    BT(S, i + 1, costo_actual)                ▶ Opción 1: no incluir el pulso i
end if
costo_salto ← 0
if S no está vacío then
    costo_salto ← costo(S.ultimo(), i)
end if
Agregar i al final de S
BT(S, i + 1, costo_actual + costo_salto)     ▶ Opción 2: incluir el pulso i
Eliminar el último elemento de S
```

Orden de complejidad Backtracking:
En cada llamado recursivo, las evaluaciones lógicas iniciales para la poda por optimalidad y la poda por factibilidad consisten en operaciones aritméticas y comparaciones simples que toman un tiempo constante O(1). Si se supera la cantidad de pulsos de la grabación (i > n) llegando al

caso base, y se debe actualizar la mejor solución encontrada, se realiza una copia del vector de selección que toma tiempo O(k).

Para los nodos internos del árbol, el algoritmo evalúa hasta dos caminos. En la rama correspondiente a excluir el pulso, las operaciones de control y el pasaje de parámetros se ejecutan en O(1). Por otro lado, en la rama correspondiente a incluir el pulso, se debe calcular el costo extra de la transición desde el último pulso agregado. Esta operación evalúa la distancia euclidiana entre dos vectores de características de dimensión d, lo que demanda un tiempo proporcional de O(d). Las acciones posteriores para insertar y extraer el pulso de la solución parcial operan en tiempo constante O(1).

En el escenario más pesimista, el árbol de recursión alcanza una profundidad n y genera un total de O(2^n) nodos. Puesto que en cada bifurcación de inclusión se realiza un trabajo de O(d) para calcular las distancias, la complejidad temporal del algoritmo queda acotada superiormente por O(2^n * d).

3.3 Programación Dinámica

La programación dinámica permite resolver el problema de manera exacta y óptima sin entrar en el costo exponencial de la fuerza bruta ni depender de la efectividad de las podas de backtracking.
El problema presenta subproblemas superpuestos. Supongamos que se seleccionaron t pulsos y el último pulso elegido fue j. Para las próximas decisiones (elegir los pulsos t+1 hasta k), no importa qué combinación de pulsos se eligió entre el 1 y el j; lo único relevante para los siguientes cortes es: (1). Cuántos pulsos ya se seleccionaron (t). (2). Cuál es el último pulso incluido (j), pues el próximo salto se medirá a partir de j. (3). Que el costo acumulado para llegar a ese estado (j, t) sea el más chico posible.
Cualquier otra secuencia de t pulsos que también termine en j pero tenga un costo mayor no puede formar parte de una solución óptima. Mientras que la fuerza bruta y el backtracking recalculan caminos que llegan al mismo estado (j, t), la programación dinámica calcula la solución óptima para cada par (j, t) una única vez y la almacena en una tabla.
El estado queda definido por el par (j, t), donde:
- $j \in \{1, \ldots, n\}$: índice del último pulso seleccionado.
- $t \in \{1, \ldots, k\}$: cantidad de pulsos seleccionados hasta el momento.
Se utiliza una matriz dp de dimensiones (n+1)x(k+1), donde cada celda dp[j][t] representa el costo mínimo de una selección válida de t pulsos que comienza en el pulso 1 y termina en el pulso j.
Cálculo de bottom up:
- Caso base: La selección debe comenzar obligatoriamente en el primer pulso. Por lo tanto, seleccionar 1 pulso terminando en el pulso 1 tiene costo cero: dp[1][1] = 0.0. Todas las demás celdas se inicializan en infinito (∞).
- Paso inductivo: El valor de dp[j][t] se construye mirando únicamente los estados de la columna anterior (t - 1): dp[j][t] = min_{t - 1 ≤ i < j} { dp[i][t - 1] + costo(i, j) }

Dado que cada columna t solo depende de los valores de la columna t - 1, la tabla se completa de manera iterativa (bottom-up) recorriendo t desde 2 hasta k.
Al finalizar el llenado, el costo óptimo de la grabación completa se encuentra en dp[n][k].

La solución se reconstruye hacia atrás comenzando en j = n para t = k. En cada paso t (descendiendo desde k hasta 2), se busca el pulso previo i que generó dicho óptimo, es decir, aquel que satisface dp[j][t] = dp[i][t - 1] + costo(i, j). Al encontrarlo, se establece j = i, se agrega a la solución y se continúa con t - 1 hasta alcanzar el pulso inicial j = 1.

PseudoCódigo Programación dinámica

```
DP(instancia)
    n ← instancia.n()
    k ← instancia.k()

    Crear matriz dp de (n + 1) × (k + 1) inicializada en ∞

    dp[1, 1] ← 0

    ► Llenado de la matriz (Bottom-up)
    for t = 2, ..., k do                ► t: cantidad de pulsos
seleccionados
        for j = t, ..., n do        ► j: pulso actual evaluado como el
t-ésimo
            for i = t - 1, ..., j - 1 do    ► i: pulso anterior evaluado
como el (t-1)-ésimo
                if dp[i, t - 1] ≠ ∞ then
                    candidato ← dp[i, t - 1] + costo(i, j)
                    if candidato < dp[j, t] then
                        dp[j, t] ← candidato
                    end if
                end if
            end for
        end for
    end for

    ► Reconstrucción de la solución óptima
    S ← [n]
    j ← n
    for t = k, …, 2 do
        for i = t -1, …, j - 1, do
            if dp[i, t-1] ≠ ∞ y dp[j,t]=dp[i,t-1] + costo(i,j) then
                  j ← i
```

```
                Agregar j al final de S
                break
            end if
        end for
    end for
    Invertir S
    return S
```

Complejidad Temporal:
El llenado de la matriz se realiza mediante tres ciclos anidados:
1. El ciclo “externo” t realiza k iteraciones.
2. El ciclo “intermedio” j realiza a lo sumo n iteraciones.
3. El ciclo “interno” i evalúa los predecesores posibles, acotado también por n iteraciones.
En el cuerpo del ciclo interno, calcular la distancia euclidiana entre dos vectores de características de dimensión d toma tiempo O(d). Por lo tanto, el llenado de la tabla toma O(n^2*k*d).
En la reconstrucción se ejecutan como mucho k iteraciones donde se busca entre a lo sumo n predecesores calculando distancias, lo que es O(n*k*d).
La complejidad temporal total queda dominada por el llenado de la tabla O(n^2*k*d). Como en las grabaciones de audio analizadas la dimensión de las características es una constante fija (d = 12), la complejidad temporal respecto al tamaño de la entrada es O(n^2*k). Si en un experimento particular se fija k = n / 2, la complejidad escala a O(n^3).
Complejidad Espacial:
La memoria del algoritmo está compuesta por:
1. La matriz dp de dimensiones (n+1)x(k+1), que requiere espacio O(n*k).
2. El almacenamiento de las características de la instancia, que requiere O(n*d).
La complejidad espacial total se mantiene en O(n*k + n*d) = O(n*k).

4. Experimentación

Instrucciones de compilación y Entorno de ejecución

Los experimentos se llevaron a cabo bajo las siguientes especificaciones:

- Procesador: AMD Ryzen AI 5
- Memoria RAM: 16 GB.
- Sistema Operativo: Ubuntu 22.04 LTS (vía WSL2 en Windows 11)
- Compilador C++: g++ versión 11.4.0 y C++17.
- Intérprete Python: Python 3.10

El proyecto incluye un Makefile para compilar las soluciones en C++.
Para compilar: make clean && make

Para ejecutar los algoritmos en C++: ./radioedit --instancia <archivo_entrada> --algoritmo <fb|bt|pd> [--salida <archivo_salida>]
Para ejecutar las versiones en Python: python3 python/backtracking.py --instancia <archivo_entrada> python3 python/programacion_dinamica.py --instancia <archivo_entrada>

4.1 Comprobación de las implementaciones

Hipótesis: Las cinco implementaciones (FB en C++, BT en C++, PD en C++, BT en Python y PD en Python) son equivalentes y exactas, es decir que ante cualquier instancia del problema, todas deben dar soluciones válidas con el mismo costo óptimo global.
Resultados: Se ejecutaron los cinco algoritmos en seis instancias de prueba de tamaño reducido (para que FB no tarde tanto). Las instancias incluyen desde el caso del enunciado (ejemplo.txt) e instancias con n entre 8 y 20, hasta un fragmento de audio real (Giveme_10s.txt con n=23 y d=12).

Instancia	Pulsos (n)	Seleccionar (k)	Costo FB (C++)	Costo BT (C++)	Costo PD (C++)	Costo BT (Py)	Costo PD (Py)	Coinciden?
ejemplo.txt	6	4	2.00000	2.00000	2.00000	2.0000	2.00000	SÍ
instancia_n8.txt	8	4	0.28536	0.28536	0.28536	0.28536	0.28536	SÍ
instancia_n12.txt	12	6	0.46365	0.46365	0.46365	0.46365	0.46365	SÍ
instancia_n16.txt	16	8	0.55395	0.55395	0.55395	0.55395	0.55395	SÍ
instancia_n20.txt	20	10	0.36087	0.36087	0.36087	0.36087	0.36087	SÍ
Giveme_10s.txt	23	11	0.51400	0.51400	0.51400	0.51400	0.51400	SÍ

Conclusión: La hipótesis se confirma. En todos los casos analizados, los cinco métodos alcanzaron soluciones válidas con el mismo valor de función objetivo. Esto demuestra que:
1- Las podas en BT no descartan ninguna rama que contenga el óptimo
2- La definición del estado y la recurrencia bottom-up de PD resuelven correctamente el

problema de optimización

3- Las implementaciones trasladadas a Python reproducen con exactitud la lógica de C++

4.2 Análisis de la complejidad

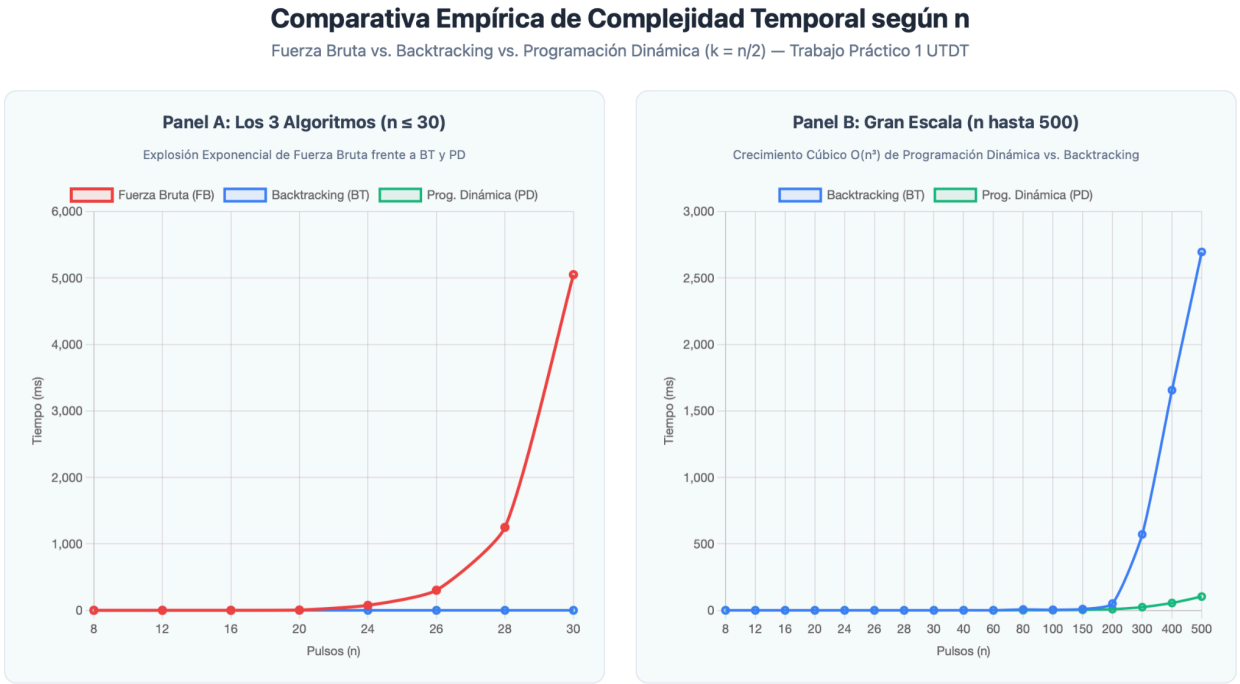
Hipótesis:

1- FB: Al explorar un árbol binario, su tiempo de ejecución escalará de forma exponencial $O(2^n)$. Por lo tanto, duplicará su tiempo por cada pulso adicional, volviéndose inviable para n>30.

2- PD: La complejidad teórica es $O(n^2 * k * d)$. Dado que la dimensión de las características es una constante (d=12) y que en este experimento se fija una tasa de recorte constante (k=n/2), la complejidad teórica se convierte a $O(n^2 * n/2 * 12) = O(6n^3) = O(n^3)$. Por lo tanto, el tiempo de ejecución crecerá de manera cúbica con respecto a n.

3- BT: Debido a las podas, será órdenes de magnitud más rápido que FB pero su tiempo dependerá de la capacidad de poda sobre cada instancia, mostrando una tasa de crecimiento menos regular que la de PD.

Resultados: Se utilizó un dataset de 15 instancias con tamaños desde n=8 hasta n=500, fijando en todos los casos la mitad de puntos a retener (k=n/2) y dimensión d=12. Cada prueba se ejecutó 10 veces en C++, registrando la media en ms. A continuación se resumen los tiempos obtenidos.



Conclusión: Los resultados confirman las hipótesis.

1- Comportamiento exponencial de FB. Para n=30, FB ya demora más de 5 segundos, y para instancias reales de por ejemplo n>100, el cálculo requeriría tiempos gigantescos, confirmando

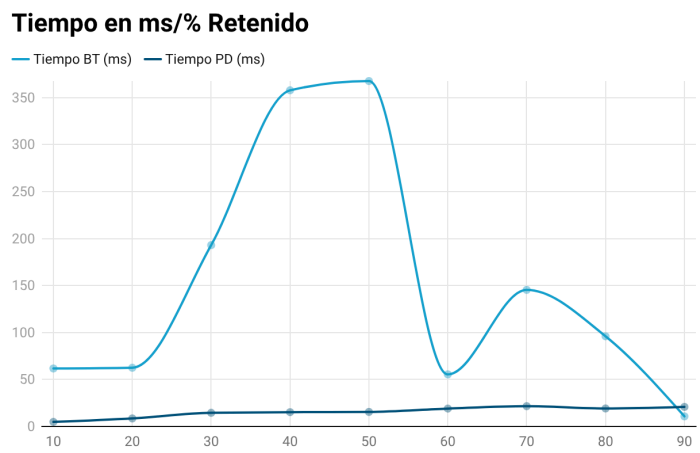
que es inviable para situaciones reales.
2- Tiempo Cúbico de PD. El tiempo de ejecución de PD crece de forma polinomial. Para n = 500, PD resuelve la instancia en 103ms, demostrando una excelente escalabilidad.
3- Comportamiento de BT: Si bien BT logró reducir en órdenes de magnitud el tiempo respecto a FB, su tiempo de ejecución para n = 500 llegó a casi 2,7 segundos (26 veces más lento que PD), exhibiendo una tasa de crecimiento con mayor dispersión.

4.3 Tiempo de ejecución frente al factor de recorte (k/n)

Hipótesis: A mayor cantidad de pulsos conservados k (es decir, ante un recorte menos agresivo donde se descarta poco audio), el tiempo de ejecución aumentará de forma constante y monótona en todos los algoritmos, alcanzando su punto más lento cuando k se aproxima a n. Esto se fundamenta en la idea de que conformar una selección más larga requiere evaluar más transiciones intermedias y almacenar una mayor cantidad de pulsos en la solución final.
Resultados: Para evaluar de forma aislada el impacto del factor de recorte, se fijó una única pista del dataset (Baby Shark, con n=222 pulsos fijos y dimensión d=12) y se varió el porcentaje de retención desde el 10% (k=22) hasta el 90% (k=200) en intervalos del 10%. Cada prueba se ejecutó 5 veces en C++, promediando los tiempos de ejecución:

Resultados de las Mediciones Numéricas

Porcentaje Retenido (k/n)	Pulsos a conservar (k)	Tiempo BT (ms)	Tiempo PD (ms)	Costo Óptimo Encontrado
10%	22	61,72	4,85	0,4917
20%	44	62,52	8,61	0,3668
30%	67	192,99	14,49	0,0099
40%	89	357,67	15,22	0,2945
50%	111	**367,54**	15,50	0,2420
60%	133	55,42	19,04	0,1211
70%	155	145,29	21,59	0,2292
80%	178	95,94	19,18	0,1893
90%	200	**10,74**	20,84	0,0969



Conclusión: Los resultados obtenidos conducen a una conclusión que refuta la hipótesis inicial para BT pero la confirma para PD.
La hipótesis de que el tiempo de BT crecería monótonamente hacia el 90% resultó falsa. En lugar de ser más lento a mayor k, el tiempo describe una curva acampanada. Alcanza su máximo tiempo en el centro. En el extremo superior, el tiempo cae a 10,7ms. Este comportamiento se debe a que el esfuerzo de BT no depende de cuántos pulsos se conservan, sino de la cantidad de combinaciones y de la efectividad de la poda por factibilidad. Cuando k es cercano a n, la condición de factibilidad es muy restrictiva y poda la rama en los niveles más altos del árbol. En cambio, cuando k es cercano a n/2, es cuando más combinaciones posibles puede haber y se descartan más, lo que obliga al algoritmo a explorar un árbol mucho más profundo antes de podar.
Para PD, la hipótesis inicial si se cumple. El tiempo escala monótona y linealmente con k. Al ser un método iterativo que calcula una matriz de nxk, la cantidad de celdas a completar es proporcional a k.

4.4 Influencia de la estructura musical del género y la densidad rítmica

Hipótesis
1- El tiempo de ejecución no dependerá de la duración del audio, sino de la cantidad de pulsos n, que está determinada por el tempo (BPM) de la canción. Pistas más lentas generarán menos pulsos por segundo y se resolverán más rápido, independientemente de su duración total.
2- En BT, la capacidad de poda por optimalidad dependerá mucho del género musical:
- En canciones de pop / electrónica, que son géneros muy repetitivos, muchos saltos tendrán costos cercanos a cero. Por ende, el costo parcial tardará en acumularse y la poda por optimalidad no podrá cortar ramas rápido.
- En géneros con más variedad como Rock o música clásica, los saltos elevan el costo euclidiano inmediatamente, permitiendo que la poda por optimalidad descarte la gran mayoría del árbol.
- El tiempo de PD dependerá de los parámetros n y k exclusivamente.

Resultados:
Se evaluaron seis canciones de distintos géneros con un factor de recorte del 50% (k=n/2), registrando la duración de tiempo original, el tempo, la cantidad de pulsos n resultantes y los tiempos de ejecución en C++.

Obra musical	Género	Tempo aprox.	Duración original (s)	Pulsos (n)	Pulsos a retener (k)	Tiempo BT (ms)	Tiempo PD (ms)	Saltos hallados

Moonlight Sonata (Beethoven)	Clásica	60	304s	236	118	62,04	12,08	1
Baby Shark (Pinkfong)	Infantil	115	131,1	222	111	282,14	9,35	3
Trátame Suavemente	Pop-Rock	118	202,4	398	199	1190,23	54,98	2
Un Ángel Para Tu Soledad (PR)	Rock Nacional	130	231,5	501	250	1093,71	102,35	2
Give Me Everything (Pb)	Dance - Pop	129	266,3	535	267	9222,87	132,52	1

Conclusión: Los resultados confirman las hipótesis planteadas. Moonlight Sonata es la obra más larga del dataset. Sin embargo, al tener un tempo lento, se divide en solo n=236 pulsos. Por el contrario, Give Me Everything dura menos tiempo en minutos, pero al ser música rápida, genera más del doble de pulsos (n=535). Como los algoritmos operan sobre n, la de Beethoven se resolvió en sólo 12 ms en PD y 62 en BT mientras que la de Pitbull tardó 132ms en PD y más de 9.2 segundos en BT.

Al comparar Un Ángel para tu Soledad con Give Me Everything, ambos tienen una cantidad similar de pulsos. En PD, la diferencia de tiempo es del 29%, lo que concuerda con el aumento de n. En cambio, en BT, Give Me Everything tardó 8.4 veces más. Las repeticiones en la pista hacen que casi cualquier salto entre estribillos tenga un costo muy pequeño, debilitando la poda por optimalidad. Lo mismo ocurre entre Moonlight Sonata y Baby Shark. A pesar de que la obra clásica tiene mas pulsos, BT fue 4,5 veces más rápido en la de Beethoven que en la canción infantil.

PD demostró un comportamiento uniforme. Sus tiempos aumentaron suavemente en función de n sin importar el género. Esto la consolida como el único enfoque seguro para audios arbitrarios.

4.5 Comparación del lenguaje de programación (C++

vs. Python)

Hipótesis: C++ supera a Python en rendimiento tanto en PD como en BT. Esta diferencia de rendimiento no dependerá de las características de una canción particular, sino que será un fenómeno producto de la compilación nativa, del tipado estático y la eficiencia en el acceso continuo a memoria.

Resultados: Para validar la hipótesis se evaluaron 5 instancias de tamaño progresivo. Desde un fragmento corto (n=23) hasta piezas completas como Baby Shark (n=222) y la obra más densa del dataset, Give Me Everything (n=535), fijando en todos los casos k=n/2.

Instancia	Pulsos (n)	Pulsos a Retener (k)	PD C++ (ms)	PD Py (ms)	BT C++ (ms)	BT Py (ms)
Giveme_10s.txt	23	11	0.04	189.33	0.04	199.99
Instancia_n50.txt	50	25	0.18	193.43	3.07	395.23
Instancia_n100.txt	100	50	2.42	361.78	5.42	629.68
BabyShark.txt	222	111	22.18	2887.99	359.99	32714.21
GiveMeEverything.txt	535	267	259.71	36816.59	9222.87	1247098.29

Conclusión: La hipótesis se confirma. Para todas las instancias, se puede ver que C++ es notoriamente más veloz que Python. Esto refuta con datos empíricos que la diferencia observada fuera una anomalía de una canción favorable, ya sea en PD o en BT, C++ supera por mucho a Python.

Algoritmo a utilizar en la práctica: En base a los resultados obtenidos a lo largo de este informe, la recomendación es utilizar Programación Dinámica implementada en C++.

Frente a FB y BT, PD garantiza la solución exacta de mínimo costo con un comportamiento temporal predecible O(n^3) con k=n/2, sin sufrir la explosión de complejidad ni la degradación de rendimiento que experimenta BT en canciones repetitivas.

Frente a Python, C++ resuelve canciones completas en menos de 260ms, lo que permite su

integración en software de edición de audio o producción interactiva en tiempo real, mientras que la misma solución en Python demandaría tiempos de espera gigantes para un usuario.

(*Para la re-implementación de los algoritmos de BT y PD en Python a partir del código base en C++, se utilizó la asistencia de Gemini. El registro de la conversación se encuentra disponible en* https://share.gemini.google/jOJTMBVzSbS4)

Conclusión

En el trabajo se analizó y resolvió el problema del recorte consciente de audio mediante tres algoritmos: Fuerza Bruta, Backtracking y Programación Dinámica. Los resultados demostraron las diferencias teóricas y prácticas de cada enfoque. Como era esperado, la Fuerza Bruta resultó inviable para audios reales debido a su complejidad. Por su parte, la Programación Dinámica se consolidó como el método más robusto, validando su complejidad teórica O(n^3) con un margen de error mínimo y tiempos de ejecución muy bajos y predecibles.

Un descubrimiento importante de este estudio fue el comportamiento del algoritmo de Backtracking, que demostró una fuerte dependencia tanto del género musical analizado como del porcentaje de recorte objetivo (k). En géneros con muchas repeticiones como el Pop, la escasez de penalizaciones tempranas en el costo impidió que las podas por optimalidad funcionaran correctamente. Además, se comprobó que la complejidad computacional del problema no está atada a la duración en minutos de la pista, sino a su densidad rítmica (BPM), que determina la cantidad de pulsos n a procesar.

Respecto al lenguaje de programación, la experimentación mostró que C++ es muy superior a Python para este tipo de problemas. Python sufrió un cuello de botella, llegando a ser cientos de veces más lento en Programación Dinámica y tardando minutos en resolver lo que C++ procesó en segundos con Backtracking.

A partir de estos resultados, la recomendación es utilizar Programación Dinámica en C++. Como se pudo ver, este algoritmo es la única opción viable, porque asegura encontrar el recorte de menor costo en tiempos muy cortos (por debajo de los 150 ms), y su velocidad no es perjudicada si se analizan canciones con estructuras repetitivas o al variar el porcentaje de audio a recortar.

Por último, es importante señalar las limitaciones que se encontraron en el modelo. La mayor limitación es que el cálculo del costo (la distancia euclidiana) no siempre coincide con lo que se percibe en la realidad. En los experimentos se vió que, aunque para el Pop o la música infantil los saltos logran ser inaudibles, en géneros como el Rock o la música clásica los cortes "óptimos" siguen siendo notorios porque hacen cortes bruscos en la voz, o la melodía. Para trabajos futuros, se sugiere modificar la fórmula del costo para que de alguna forma logre identificar e ignorar los cortes en el medio de las voces o los instrumentos principales, llegando así a que las transiciones suenen naturales en todos los géneros.